Análise de Dados com OpenMP

Coeficiente de variação para estatura e peso em dados agrupados

Instituição: IESB · Curso: Ciência da Computação
Aluno: Heitor Ribeiro · Matrícula: 2412130143
Disciplina: Programação Paralela · Turma: _____
Professor: Rodrigo Gonçalves Pinto

Medição analisada: 12/11/2026 17:41 -03:00 · Dados sintéticos · N = 1.000.

1. Objetivo e conjunto de dados

O exercício propõe calcular os coeficientes de variação da estatura e do peso de 1.000 pessoas, usando dados agrupados e OpenMP. Foram implementadas duas rotas completas em C++17: uma sequencial e outra paralela, com conferência de corretude e comparação dos tempos de processamento.

A base é inteiramente sintética, gerada com semente 20261112. Cada linha contém estatura em centímetros e peso em quilogramas, sem cabeçalho. Os primeiros dez pares incluem fronteiras exatas das classes. A base não representa uma coleta de pessoas reais nem permite inferências demográficas.

2. Agrupamento e definição das estatísticas

O enunciado fixa amplitudes de 8 cm para estatura e 4 kg para peso, sem determinar a origem. Adotou-se início = floor(mínimo/amplitude) × amplitude, separadamente para cada variável. As classes são semiabertas [a, b): um valor igual a b pertence à classe seguinte. O ponto médio é x_i = a + amplitude/2.

Se f_i é a frequência da classe i e k é o número de classes, então $N = \Sigma_i f_i$. Para i = 1, …, k, calculam-se a média agrupada μ, a variância populacional σ^2, o desvio padrão σ e o coeficiente de variação percentual:

$$\mu = [\Sigma_i (x_i \times f_i)] / N$$

$$\sigma^2 = [\Sigma_i f_i \times (x_i - \mu)^2] / N$$

$$\sigma = \sqrt{(\sigma^2)}$$

$$CV\ (\%) = 100 \times \sigma / \mu$$

A variância usa divisor N. A média é ponderada pelos pontos médios e frequências, em vez de ser a média dos valores individuais. Utilizam-se duas passadas: a variância depende da média já concluída. Esse método evita a subtração de termos próximos da expressão alternativa $E[X^2] - E[X]^2$.

Distribuições de frequência

A estatura observada varia de 145,0 a 199,9 cm, em sete classes com origem em 144 cm. O peso varia de 44,0 a 109,5 kg, em 17 classes com origem em 44 kg. As duas somas de frequências são exatamente 1.000. As tabelas são extraídas da saída C++; nelas, mantém-se o ponto como separador decimal.

Estatura (cm)

Intervalo [a, b)	Ponto médio x_i	Frequência f_i
[144.0, 152.0)	148.0	15
[152.0, 160.0)	156.0	94
[160.0, 168.0)	164.0	276
[168.0, 176.0)	172.0	334
[176.0, 184.0)	180.0	198
[184.0, 192.0)	188.0	66
[192.0, 200.0)	196.0	17

Peso (kg)

Intervalo [a, b)	Ponto médio x_i	Frequência f_i
[44.0, 48.0)	46.0	24
[48.0, 52.0)	50.0	20
[52.0, 56.0)	54.0	47
[56.0, 60.0)	58.0	58
[60.0, 64.0)	62.0	79
[64.0, 68.0)	66.0	114
[68.0, 72.0)	70.0	130
[72.0, 76.0)	74.0	140
[76.0, 80.0)	78.0	107
[80.0, 84.0)	82.0	103
[84.0, 88.0)	86.0	67
[88.0, 92.0)	90.0	53
[92.0, 96.0)	94.0	31
[96.0, 100.0)	98.0	14
[100.0, 104.0)	102.0	10
[104.0, 108.0)	106.0	2
[108.0, 112.0)	110.0	1

3. Implementação e paralelização

A rota sequencial executa extremos, construção de classes, frequências e estatísticas sem chamadas OpenMP no processamento. A rota paralela realiza o mesmo trabalho lógico com as estratégias resumidas a seguir.

Etapa	Estratégia OpenMP e segurança
Extremos	parallel for com reduction(min:...) e reduction(max:...).
Pontos médios	parallel for schedule(static), com um escritor por posição.
Frequências	Histograma privado por thread; combinação por classe após barreira.
Média	parallel for reduction(+:soma_ponderada), acumulando $x_i \times f_i$.
Variância	Segunda redução, acumulando $f_i \times (x_i - \mu)^2$ com μ já concluída.

Os dados de entrada são somente lidos pelas threads. Os histogramas privados evitam incrementos concorrentes na mesma frequência, e as reduções combinam acumuladores locais. Não se usa critical ou atomic por registro. As barreiras garantem as dependências; schedule(static) é adequado ao custo uniforme das iterações. Não há paralelismo aninhado nem divisão manual por IDs fixos.

Cada processamento das duas variáveis abre dez regiões paralelas. O ajuste dinâmico é desativado e o número real de threads é consultado dentro da equipe.

4. Resultados estatísticos

Estatística	Estatura	Peso
Média agrupada	170.9760000000 cm	72.8400000000 kg
Variância populacional	88.6794240000 cm²	146.5584000000 kg²
Desvio padrão populacional	9.4169753106 cm	12.1061306783 kg
CV	5.5077761268%	16.6201684216%

Os resultados conferem com a referência estatística fornecida. Frequências, limites e pontos médios coincidem exatamente entre as versões; as estatísticas são comparadas com tolerâncias absoluta e relativa de 1e-10. O executável calcula a partir do TXT, sem ler valores esperados. O peso apresenta maior dispersão relativa neste conjunto sintético agrupado.

5. Comparação de desempenho

Ambiente e método

Ambiente: Windows-11-10.0.26200-SP0; CPU 12th Gen Intel(R) Core(TM) i5-1235U; 12 processadores lógicos. Compilador: g++.EXE (Rev3, Built by MSYS2 project) 16.2.0.

```
Flags: -std=c++17 -O2 -Wall -Wextra -Wpedantic -fopenmp
```

Há cinco processamentos de aquecimento por configuração, fora do cronômetro. O benchmark usa omp_get_wtime na thread principal, com 10 amostras de 1000 processamentos completos por configuração. Divide-se cada lote por 1000 para obter o tempo por processamento. Mantêm-se os mesmos 1.000 pares e o mesmo número de repetições em todas as versões. A ordem das configurações alterna a cada rodada, e a corretude é conferida após cada amostra.

O tempo inclui extremos, classes e suas alocações, histogramas, conferência dos totais, estatísticas das duas variáveis e liberação dos resultados. Inclui-se um pequeno checksum volatile comum às rotas para tornar a computação observável; não se usa LTO. Leitura de disco, impressão, testes e escrita dos CSVs ficam fora da medição. A rota OpenMP com uma thread é medida separadamente da versão sequencial real.

> Speedup = mediana sequencial / mediana paralela
>
> Eficiência = speedup / número real de threads

Tempos observados e interpretação

Medianas por processamento completo, em microssegundos (µs). A eficiência é expressa em percentual. Todos os valores provêm da mesma medição registrada.

Versão	Threads	Mediana (µs)	Speedup	Eficiência (%)
sequencial	1	12.000	1.000000	100.0000
paralela	1	60.000	0.199999	19.9999
paralela	2	552.000	0.021739	1.0870
paralela	4	1016.000	0.011811	0.2953
paralela	8	1740.000	0.006897	0.0862

Todas as configurações paralelas tiveram mediana maior que a sequencial. A versão sequencial apresentou a menor mediana. Para esta entrada pequena, os custos de regiões paralelas, barreiras, reduções e combinação dos histogramas superam o trabalho distribuído. Os laços estatísticos têm apenas sete e 17 classes. Essa interpretação explica os tempos completos; os custos individuais não foram medidos separadamente.

Não se fixou afinidade nem se controlou exclusivamente carga, temperatura ou política de energia. O escalonamento da CPU híbrida pode variar. Os resultados descrevem esta execução, sem generalização para outras máquinas. Não se aumentou N artificialmente para buscar ganho; repetir a mesma base não amplia cada processamento.

6. Validação, rastreabilidade e conclusão

O caso manual de estaturas [145, 145, 153, 169] produz pontos médios [148, 156, 164, 172], frequências [2, 1, 0, 1], média 156 e variância 96. Os testes também cobrem fronteiras exatas e valores adjacentes em double, classes vazias, N = 1, dados constantes e fronteira como máximo. Foram registradas 400 comparações sequencial/paralela e 47 verificações funcionais, incluindo entradas inválidas e a integridade da base e da referência.

No Windows, os testes verificam ainda os runtimes locais e o início dos executáveis com PATH restrito ao sistema. Os scripts de diagnóstico propagam códigos de saída e mensagens completas. Os registros detalhados ficam em resultados/testes.json; a auditoria histórica permanece separada dos testes funcionais.

As 50 amostras individuais e as medianas estão em resultados/amostras.csv e resultados/benchmark.csv. O registro benchmark_execucao.json associa a execução ao executável, às fontes, à entrada e ao ambiente por hashes, conferidos antes e depois da medição. O relatório recalcula os indicadores dos CSVs e rejeita registros divergentes. A revisão documental preserva essa medição e não executa um benchmark novo.

Conclui-se que os CVs são 5.5077761268% para estatura e 16.6201684216% para peso. A implementação cumpre o agrupamento e a paralelização solicitados com verificação de corretude. A versão sequencial foi mais rápida no conjunto analisado, evidenciando que paralelizar uma carga pequena não garante ganho.

7. Reprodução

No Git Bash, na raiz do repositório:

```
python -B ferramentas/compilar.py
./build/openmp_cv.exe dados/dados_1000_pessoas.txt --threads 4
python -B testes/validar.py
python -B ferramentas/medir.py --amostras 10 --repeticoes 1000
python -B ferramentas/gerar_relatorio.py
```

Para revisar somente os documentos, preservando a medição atual:

```
python -B ferramentas/gerar_relatorio.py --usar-registros
```

O README apresenta todos os testes, as dependências e o procedimento Windows. O C++ direto produz somente CSVs; medir.py registra também os metadados de integridade necessários ao relatório. Fontes TrueType do sistema são incorporadas ao PDF; não são necessários pacotes Python adicionais.

Referências bibliográficas

PINTO, Rodrigo Gonçalves. Exercício Final: Análise de Dados com OpenMP. IESB, 28 out. 2025. Enunciado acadêmico fornecido pelo professor, duas páginas.

PINTO, R. G. Parallel Programming Examples in C++ using OpenMP. 2025. Material didático fornecido pelo professor: ambiente (001), laços (004), tempo e barreiras (005), sincronização (006), reduções e duas passadas (007) e tarefas (008). As técnicas foram estudadas e a solução foi escrita para este exercício. Os materiais são referências bibliográficas, sem constituir dependências da entrega independente.